

What is `yield` in Python?

In Python, `**yield**` is a keyword used inside a function to turn it into a generator.

Theoretical Definition:

A generator is a special type of function that **produces a sequence of values over time**, instead of returning them all at once like a regular function. This allows for memory-efficient processing of large data sets.

How does `yield` work?

When Python encounters a `yield` statement in a function:

- It **pauses** the function's execution.
- **Returns** the value to the caller.
- **Saves the function's state**, so it can resume from where it left off (using `next()` or a `for` loop).



Simple Example:

python

Copiar

Editar

```
def count_up_to(n):  
    for i in range(n):  
        yield i  
  
generator = count_up_to(3)  
  
for number in generator:  
    print(number)
```

Output:

Copiar

Editar

```
0  
1  
2
```

Comparison with `return`:

Feature	<code>return</code>	<code>yield</code>
Returns	A single value, ends the function	A value and pauses the function
Can be resumed?	 No	 Yes
Common use	Regular functions	Generator functions

When to Use `yield`?

- When working with **large datasets** and you want to avoid loading everything into memory.
- When you need **on-demand** (lazy) value generation.
- When building **processing pipelines** or **custom iterators**.